Grant Guide App

Danny Guo

guo3105@vandals.uidaho.edu

University of Idaho

Dylan Gomez

gome2670@vandals.uidaho.edu

University of Idaho

Abstract

This paper lays out the foundation and process of creating a grant budgeting app called Grant Guide. The project is created for the Database Systems class at the University of Idaho and shows our understanding for the project and its end goals. Our Grant Guide application will allow researchers to easily form multi-year budgets in accordance to federal and state guidelines.

To make Grant Guide, we will use HTML/CSS/JavaScript for our front end. For our database, we plan on using MySQL and this will store all of our tables and data needed for the calculations. For the back end, Node.js will be used since Javascript will also be used in the front-end. Keeping the same language between front end and back end will be helpful to maintain readability and communication. The purpose of this program is to help grant builders construct valid grants in a faster and more organized manner.

Keywords

Entity Relation Model, Principle Investigator (PI), Budget Application, Database

CCS Concepts

- Information systems → Database management system engines
- Human-centered computing → Interactive systems and tools
- Applied computing → Decision support systems
- Software and its engineering → Web-based software

1 Introduction

Currently, when designing a budget for a grant given by the National Institutes of Health (NIH) and the National Science Foundation (NSF), there are many restrictions and minute details to know. This makes the budgeting process slow and inefficient, which can lead to grants being delayed, rewards being reduced, or even the grant being denied.

Principal Investigators (PI) are using spreadsheets to compose budgets that are timely and error prone. Our budgeting app, called Grant Guide, will simplify the process and streamline the information. We will have a step by step process in which the user can visualize the budget and have the application auto-generate a compliant budget.

The main details of Grant Guide are that a researcher can look at our interface and enter in specific information for a budget. The first step is to decide how long the budget will run for; our app allows up to five years in advance. Then, the user will decide who is included in the budget, such as who is the PI. This could be themselves or another person if someone else is creating a grant for someone else. Other roles included in the grant are co-PIS, staff, and students.

Afterwards, the PI will enter the FTE rate (Full-Time Equivalence) of all the participants to calculate their salary for that year. Some limitations apply since students can only have an FTE rate of 50% or 20 hours a week. Our application will verify the user input to make sure that for students, FTE rates can't go above 50%.

Next, travel information is entered in, specifically whether the trip takes place domestically or internationally This allows for the database to use the right rates when calculating travel costs. Other important information includes the number of people going on the trip and how long the trip will last.

Lastly, any sub awards are entered into the application. These sub awards has to follow the same guidelines and restrictions that the main budget does before it gets to recipients and this will be checked by our database.

An integral part of our design will be real time data validation from the user. Examples of this include checking to see if FTE rate of a student is over 50% or over 100% for a faculty member. It will display a error message, and let the user know that the number they entered does not fall within guild lines.

2 ER Model

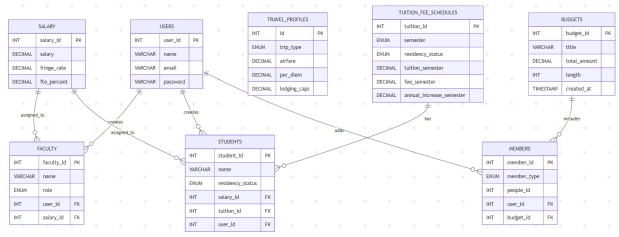


Figure 1: ER Diagram

Our ER diagram captures the database design and workflow for how Grant Guide stores and communicates data to other tables. Our entities will be Users, Members, Salary, Students, Faculty, Tuition and Fees, Budget, and Travel.

Users: The users table will store the user information when they register an account to use our website. Name, email, and password will be the attributes with id being the primary key for the table.

, ,
2025.

The password will not be stored in plain text, but is hashed before it gets inserted.

Members: The members table will contain user id, budget id, member type, people id, and member id as the primary key. This table is used to show what members are apart of what grants created by a certain user.

Salary: The Salary entity contains the attributes: fringe rate, FTE%, salary, salary id as the primary key. The salary entity will apply the fringe rate and FTE percentage to the Salary. Depending on the role, the FTE percentage will shift.

Student: The student entity contains the attributes: name, residency status, salary id as a foreign key, and student id as the primary key. This will allow students to have a many to one relationship with the salary entity.

Faculty: The faculty entity contains the attributes: name, role, salary id as a foreign key, and faculty id as the primary key. Salary id will allow different percentages to be applied based on role, since three roles can be listed under faculty.

Tuition and Fee Schedules: This entity contains residency status and semester as the primary key. We will also have attributes for annual increase semester, fee semester, and tuition semester. The student entity will look at this table to figure out their tuition cost.

Budget: The budget entity contains the budget id as the primary key. Other attributes are title, total amount, length, and date created.

3 Tools

We use a standard web stack:

- **Front end:** HTML/CSS and Javascript
- **Back End:** Node.js
- **Database:** MySQL for our Database and use API calls to go from front-end to database and back end.

Development occurs locally on **XAMPP** server.

3.1 Login/Registration

The first thing the user has to do before using the application is to create an account and login. If the user tries to add someone to a grant before creating an account, an error message will up pop saying to sign in first before getting started. In the navigation bar at the top, there are only two buttons that someone can click: login and Grant Guide, with the latter bringing the user to the grant fill out page.

If its the user's first time using the app, they can click the "Create an Account" button which will bring them to the registration page. Here, they need to fill out four fields: name, email, password, and confirm password. There are requirements for each of the different fields, so there is a lot of user validation and checking when an account is created.

The first and most obvious requirement is to have no blank fields. Even if the other fields are filled out incorrectly, if there is a blank

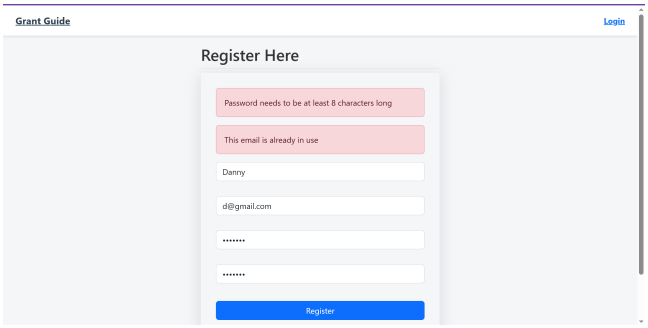


Figure 2: Registration Page

field, the application will only alert the user to fill everything out first. Secondly, one email can only have one account so if the email already exists, it will not let another account be created.

The last two validation checks have to do with the password. First, the length has to be greater than or equal to eight characters. A longer password is harder to guess and is more secure, specifically if it's a mixture of numbers, characters, and symbols. The other check is see if the value entered in the password field matches the value entered in the password confirm field. Since the user has to enter the same password twice, this will prevent typos as then the passwords won't match. This adds an extra level of assurance and security.

Once the user is successfully created and added into the database, they can navigate back to the login page to sign into the program. Here, they need to fill out two fields: email and password.

4 Project Details

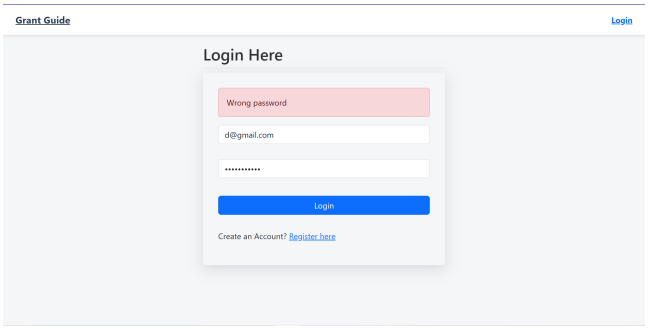


Figure 3: Login Page

If the email that is entered does not exist for an account, it will show an error message saying that the entered email is not in used. If the email does exist, then the password must be entered in correctly as well or it will reject the login. Since the password stored in the database is not in plain text, since that would be extremely insecure if someone managed to access it, our program takes the entered password and hashes it with the same algorithm. It takes the hashed password from the database and compares to see if it matches the newly hashed entered password. If they both match, a successful login will occur.

Once logged in, it will redirect the user back to the grant page where they are ready to start filling information out. In the upper right of the navigation bar where the login in button used to be, it is now replaced with a greeting and a log out button beside it.

Everything the user does is associated and linked to this particular account and the information entered will be stored in the database for future use and reference. Even if the user leaves the website and comes back another day to work on it, everything will still be available for usage.

Once the log out button is pressed, the user is brought back to the sign in page and the login button returns back in the navigation bar to indicate that no one is signed in. The grant page is cleared of any previous information and it resets as a clean slate.

4.1 Filling out the Grant

We decided to split the grant into different expenses that allows the user to focus on one group at a time. This allows for a seamless and step-by-step process, following a logical path to completion. There are seven different cards available: Proposal Setup, Personnel & Students, Travel, Tuition & Fees, Sub awards, 5-Year Planning, and Adjust Budget. We did not want to have a header bar that the user had to click to get around because that felt too disorganized. If there was a separate page for each of the expenses, then the user would have to bounce around each web page and keep track of what they entered already. This would slow down the process since each webpage would have to load every time and there is no sense of direction in terms of what to fill out first.

5 Project Details

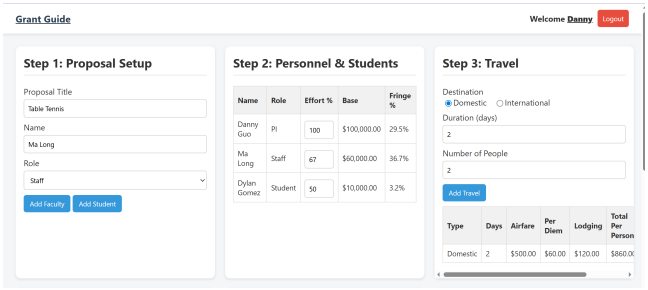


Figure 4: Front End UI

Step 1: Proposal Setup
The user can enter a name for their grant. Afterwards, they can add students and faculty into the grant by typing their name in, selecting what role to assign them, and clicking the corresponding button. However, there are some flaws to this design right now. The role option only applies if the user wanted to add faculty to the grant; they can choose from PI, Co-PIs, and staff. If the user wanted to add students, then the role they get assigned doesn't matter. They can have the role be anything as long as click the add student button. For phase three, this will be changed to have only one button called add person, and then have the selection based on what role they choose so it works for both student and faculty.

Step 2: Faculty and Students
Once people are added into the grant, they automatically appear in step 2 with some information associated with them. All the roles have differing amount of salaries, fringe benefit %, and FTE. These values are found in our database and assigned to the faculty and students based on what role was given to them in step 1. There is a small type box that allows the user to give each of the members their FTE rate, with students not allowing to go past 50%. Other checks include not being able to go above 100% or below 0%. The fringe rate percentage was found on the University of Idaho's website and that was what we used.

Step 3: Travel
Travel information is filled out next. If included, the user will decide whether the flight is going to be international or domestic, how many days will the trip last, and how many people are going on the trip. An error message will pop up if the user tries to submit a trip without selecting where you are traveling first. Once all the information is entered, it will add the trip to the table below with information on the type of trip, what many days it will last, how many people will be going, how much is the airfare, cost per person, and total cost of the trip. This allows for a convenient place for the summary of the trip.

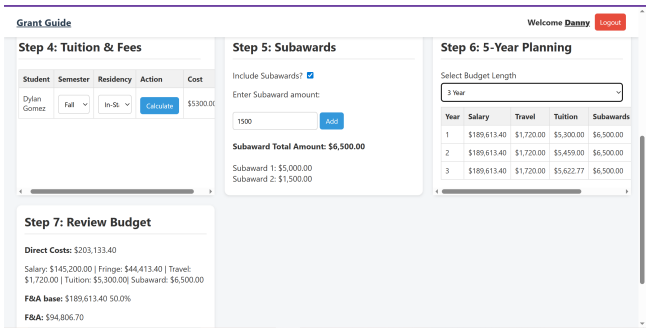


Figure 5: Front End UI pt.2

Step 4: Tuition & Fees
This card has a table that shows the students the user adds into the grant. From this table, it shows a few categories: name, semester, residency, action, and cost. From the residency column, in-state or out-state status can be selected along with the semester they are attending. Once, the calculate button is clicked, it will get the correct tuition amount from the database and assigns it to the student. Every semester has a different rate along side whether the student is in-state or out-of-state.

Step 5: Sub awards
The sub awards card only contains a checkbox at first. If the grant doesn't have any sub awards to give out, then this can be skipped over. However, if sub awards are included, then the box can be checked and a box will appear, asking how much the sub award will be. The user can enter in multiple amounts and it will list them out below the box. This card will also keep track of how much the total sub award amount is and display that above all the individual awards. If the user decides that they don't want to have

sub awards anymore, they can uncheck the box and it will remove all the amounts.

Step 6: 5 Year Plan
The five year plan allows the user to select how long they want the grant to last for. Choosing from a drop down menu, years 1 through 5 can be selected. By default, it will generate a table for one year, since that is the minimum length of the grant. Inside the table, it keeps track of six different values: years, travel cost, tuition cost, sub awards, and the total cost for that year. The purpose of this card is to make the grant for one year and then have it calculate the following years based on the rates increases found in the database.

Step 7: Review Budget
Our last card is the Review Budget. This card doesn't require any input from the user as it is a summary of the entire grant. It displays information from all the other cards in a simple format so it is easy to understand what makes up the budget. This automatically calculates the amount needed for payroll by using the FTE rate entered in by the user and the salary base gotten from the database. In addition, this also displays the Facilitates and Administrative (F&A) rate. From the direct costs, which is made up of every expense in the grant, the F&A rate doesn't apply to travel, tuition, or sub award costs. These costs have to first be subtracted from the direct total and then the difference is multiplied by our F&A rate of 50%. There is two buttons at the bottom that the user can press: Save Draft and Export to Excel.

Save Draft will save the currently entered information into the database. An alert will pop up confirming that the grant has been saved successfully to let the user know. The other button is the Export to Excel. This will create a spreadsheet that will be downloaded and contains information about the grant. Included in the spreadsheet are the following:

- Members of the grant, along with their wages, FTE, and fringe percentage.
- Travel details like trip type, number of days traveling, number of people, etc. This is only included if trips are included in the first place.
- Tuition costs of each student as well as their semester and in-state/out-state status.
- Individual sub award amounts if present.
- Grant length and the total amount for each year, accounting for the increase of tuition every year for students.

  Below these two buttons is a link to a page where the user can see all the grants they created. If this link is clicked when no one is logged in, it will display an error message asking them to do so in order to see any grants.

5.1 Reviewing Grants

5.2 Back End Design

For the back-end design, we had to quickly adapt to working with JavaScript and the Node.js environment. Coming into this project,

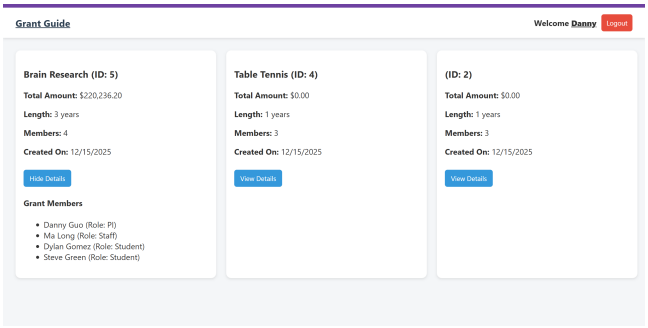


Figure 6: Front End UI pt.2

I never would have guessed just how much more coding would be involved in this section compared to both the database and front-end development. The back end required us to understand the data-flow but also understand how the server communicates between the user interface and the database. Once we laid down the foundations for our code-such as setting up the server, configuring some simple routes, and establishing those first database connections, much of the remaining work became easier. Many of the functions followed the same similar pattern with some minor tweaks, allowing us to expand our logic efficiently as this project progressed.

Our current back-end design consists primarily of server-side code written in JavaScript using Node.js to handle requests and responses between the database that was constructed using MYSQL. The back end is responsible for getting faculty, student and travel data, running basic SQL queries, and sending JSON-formatted results to the front end. This allows for our points of connection to the front-end and ensures that the data displayed in the user interface are always the same as what is stored in our database.

In the future, several important enhancements to the back-end design are planned. One of our main goals is to implement some new routes and logic for sub awards, which will be directly connected to the budget table. We will also plan to configure the budget generation button so that it automatically adds all necessary data, performs the yearly increase in costs, and outputs the final results into a downloadable Excel spreadsheet.

Before these additions take place. the front end needs to include a drop-down menu that allows the user to specify how many years the budget will cover. Once this feature is in place, the back end will calculate an annual cost increase of about 3 percent per year to reflect realistic financial growth that takes place over time. This will ensure that users plan to budgets into account for inflation and rising costs rather than underestimating their expenses.

Finally, we intend to touch up the Excel functionality. While the current implementation successfully outputs data from several of our existing tables, our goal is to have the exported sheet mirror the official budget spreadsheet that is posted on Canvas. This will make the output more professional and directly use our interactive front end calculations.

Overall, our back-end system is the concrete structure that this project relies on, and while it took some time to understand all of the moving parts, it has been very valuable in helping our front-end and back-end.

5.3 Database Design

For our database, we closely followed the structure outlined in our ER Diagram. Although it took us a considerable amount of time to piece together the diagram, it ultimately laid the groundwork for our future of coding the database effectively. Since this was the first database we had ever built from scratch, our main goal was to ensure that all relationships between tables were clearly defined and properly connected. Once we established a strong schema, populating the database was easy.

Our database currently includes tables for salaries, faculty, students, tuition and fee schedules, travel profiles, and budgets. While, this may seem like a large number of tables(at least for us it did), the separation of data helps maintain organization and prevents our information from becoming convoluted or redundant. We split our tables into two different groups, reference tables and dependent tables.

For the dependent tables, these were tables that are empty when the database is first initialized. We wanted the user to be charged of deciding what goes into these tables and be able to add information that would be relevant to the grant. Our three dependent tables are students, faculty, and budgets. The user can add either faculty and students into the database to use them in the grant and then finally save the grant into budgets to store it as an entry.

For the reference tables, these were pre-set with information when the database is initialized. This information would not come from the user and instead, have the user use the information to do something else. These tables include salary, travel profiles, and tuition and fees. If we were to have the user input these information such as fringe % rates, flight fares, or F&A rates, a number of problems can go wrong. The user can enter the wrong numbers or forget to enter them in at all and it will throw the calculations off. Furthermore, the amount of information the user has to retain about all these rates and numbers would be too much and its easier if it was already in the system for them to use/select.

Looking ahead, we plan to make several enhancements to the database. Incoming changes will include connecting the tuition and fees table to the budget table, connecting sub awards into the budgeting system, and refining some of the existing budget fields to improve usability. Additionally, one improvement noted during our class presentation was the issue of having two students with the same name entered into our front end. We plant to address this by adding unique identifiers to prevent confusion. Overall, our database design has proven to hold well over time. This is because much of it was carefully planned through the ER diagram early on.

6 Grant Page

The Grant Page serves as the centralized dashboard where users can view, manage and review all grants they have created within

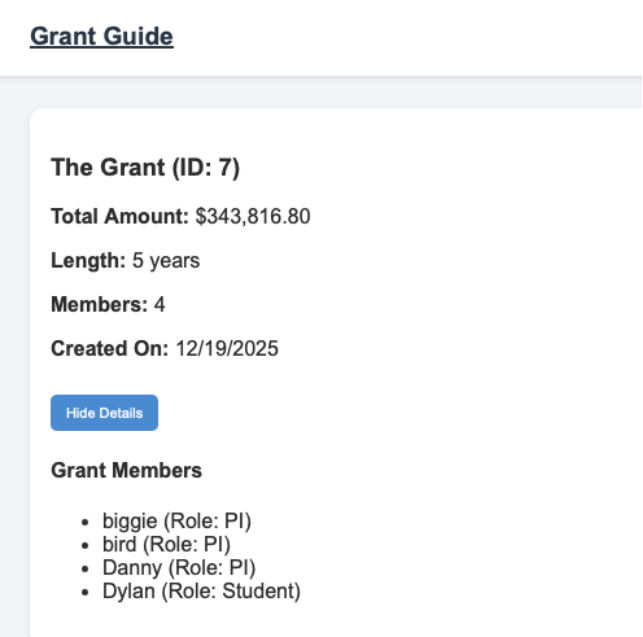


Figure 7: Grant Page

the system. Upon loading the page, the application first verifies that the user is logged in if not then they will be re-routed to the register page. Once authenticated into the system the system then retrieves all grants associated with the logged-in user and dynamically renders cards on teh page. Each grant card displays information about the grant that includes, the proposal title, total project amount, project length, number of members, and creation date, allowing the users to quickly compare and identify what grants they have already made.

To improve usability and reduce visual clutter, detailed grant information is hidden by default when loading into the page. Each grant card includes a View Details button that allows users to toggle additional information on deman. When selected, the application fetches the associated grant members from the database and displays them directly beneath the grant card without requiring a page refresh. This user interaction is handled automatically, ensuring a smooth experience while minimizing unnecessary data loading. By separating the Grant Page from the index page it gives the user the chance to reflect on the grant they made and ensure that they have everything they need. To load into this grant page the user must first fill out what members are apart of the grant, a title name, make sure they are logged in, and then finally hitting the save grant button and clicking the take me to the grants button

7 Excel Exporting Design

A major component of the Grant Guide system is the ability to export the completed budget into a structure Excel spreadsheet that closely resembles the format used in real-world grant submissions. From the beginning of development, we recognized that many researchers ultimately need something easier for them to create a

1	Title:	Im the biggest boy				
2	.					
3						
4	Personnel Compensation					
5	Name	Role	Effort	Salary	Fringe	
6	Steve	PI	21%	21000	$6,195.00	
7	George	Student	27%	2700	$86.40	
8	Dylan	Student	37%	3700	$118.40	
9	Personnel Subtotal			27400	$6,399.80	
10						
11	Travel					
12	Type	People	Days	Description	Total	
13	International	1	1	Airfare + Per Diem + Lodging	$1,445.00	
14	Travel Subtotal				$1,445.00	
15						
16	Graduate Student Tuition & Fees					
17	Student	Semester	Residency		Amount	
18	11	fall	in_state		$5,300.00	
19	12	fall	in_state		$5,300.00	
20	Tuition Subtotal				$10,600.00	
21						
22	Consortia / Subawards					
23	Subaward 1				$500.00	
24	Subaward Subtotal				$500.00	
25						
26	Total Direct Costs				$46,344.80	
27	Indirect Costs (50.0%)				$16,899.90	
28	Total Project Cost				$63,244.70	
29						
30	5-Year Budget Planning					
31	Year	Salary	Travel	Tuition	Subawards	Total
32	1	33799.8	1445	10600	$500.00	$46,344.80
33	2	33799.8	1445	10918	$500.00	$46,662.80
34	3	33799.8	1445	11245.54	$500.00	$46,990.34
35	4	33799.8	1445	11582.9062	$500.00	$47,327.71
36	5	33799.8	1445	11930.39339	$500.00	$47,675.19

Figure 8: Enter Caption

budget with but at the end of the day they would still like an excel finished product. Manually recreating a budget in Excel after using a web-based tool would defeat the purpose of automation, so the Excel export feature was designed to be seamless into the process for the user that folows the workflow.

The exported spreadsheet mirrors the logical structure of the application itself, with clearly defined sections for personnel compensation, travel, tuition and fees, subawards, and indirect costs. Each section is visually separated using and consistent with column layouts, allowing users to quickly scan and understand how individual cost categories contribute to the overall budget. By aligning the spreadsheet layout with the step by step process used in the interface, users can easily verify that the exported data matches what they entered in the system, reducing the errors and confusion with any problems associated.

Special attention was paid to ensuring that numerical values are represented correctly in the spreadsheet that we have created. Counts s-uch as the number of people and days are displayed as plain numbers, while financial values are formatted as currency, improving clarity and the look of the spreadsheet. Subtotals are calculated for each major section, and the spreadsheet clearly distinguishes between direct costs, indirect costs, and total project costs. This separation reflects common grant and helps the users understand the breakdown of expenses factoring into the final budget.

In addition to summarizing the current budget the Excel export includes a multi-year planning section that reflects the selected project duration from the user. This section provides a year-by-year breakdown of projected costs, including tuition increases over time, and gives users a forward-looking view of how their budget could change within the grant period. Including this information in the spreadsheet allows the users to perform additional analysis, adjustments, or planning without needing to return to the interface.

Overall, the Excel export feature reinforces the primary goal of Grant Guide which is reducing the effort it takes while maintaining accuracy and clarity. By producing a organized spreadsheet that is immediately usable for real grant submissions, the system bridges the gap between interactive budget creating and formal requirements. This functionality not only improves how usable are interface is but also increases the practical value of this tool that researchers can use for complex multi year grants involving lots of numbers and people.

8 User Experience Considerations and Design Rationale

A significant amount of time spent on this development effort for Grant Guide was spent designing an interface that guides users through the budgeting process in a clear and intuitive way. From the first time we read the prompt on the project we understood that researchers who prepare budget grants probably had a hard time completing one because you have to manually type a lot in. This can get confusing, and that's why we stepped into this task. We saw a way to improve this through a multi-step wizard user interface that is easy to follow. We want to reduce that mental load, prevent the confusion associated with tasks, and structure the workflow so that each step naturally leads into the next. This interface should be easy to follow, even for users who may be unfamiliar with the aspects of budgeting tools.

To accomplish this, we organized the entire interface as a one page, vertical sequence of cards, with each card representing a major category of the budget. This layout mirrors how people intuitively look at a page when reading in order. Since the user never has to leave the page they can see the whole process in one glance following the natural flow of the page. This design choice was huge because it is easy to become lost on a website when there is multiple pages and buttons everywhere.

During the first presentation of watching other people present we noticed that it was hard to track what was going on when they were maneuvering between the pages. By keeping everything on one page, it allowed users to scroll naturally, and create a sense of continuity. In many ways, this design is inspired by how spreadsheet-based grant tools are already implemented, with being able to see everything in one place, and the user can freely move between the sections without losing where they are. Even though the implementation mimics this it is less overwhelming which is a big thing with spreadsheets.

User input was another major area where we articulated niche design choices. Rather than just giving users large text boxes to hack away at the grant, we relied heavily on dropdown menus, radio buttons, and constrained numeric fields. This both simplifies the process and prevents invalid or inconsistent data that could happen by mistyping. For example, residency status is limited to in-state or out-of-state which ensures the tuition logic always receives predictable input from the user.

Travel entries must include a positive number of days, and subaward amounts can't be negative. These restrictions are not just for

usability, but they also protect the accuracy of backend calculations, which depend on data that is clean and formatted.

One of the most important validation features is the automatic enforcement of the effort percentages. As much as the user can try they cannot exceed a 50 percent FTE, and faculty cannot exceed 100 percent. This is because Students can not work full time, and often have a limit of 20 hours a week compared to the traditional 40 hours a week. Rather than waiting until the end of the process to warn the user, the interface can provide this immediate feedback by preventing these invalid values from even being entered. This approach involves the system actively guiding users toward inputs that comply with institutional rules . These rules or negations of input lower the risk of miscalculating and help maintain a final budget that is compliant with the constraints for this project.

Visual appeal was another priority throughout development. We were lucky enough to learn some appealing features through the lab, one feature called the card stood out to us when we started this project. We generally felt that cards would help us display all the steps in a organized manner. This would also help split up the tasks the user needs to do so it isn't so overwhelming. Without us even thinking about it, it also helped us add stuff to the cards because it was easy to spread out in our html file, while also allowing us to tone down the css usage. Additionally, each table shows only the most essential information needed at that stage of the process, preventing clutter and helping the user focus. After the user has inputted everything the last card shows all the costs-salary, fringe, travel, tuition, subawards, and F&A into a single, easy to read breakdown. This helps the user double check their process before exporting over to a excel spreadsheet.

Looking forward to our last touches of the project, there is some room to refine and enhance the user interface based on continued testing and feedback from classmates and friends. Planned things to change before the final pitstop include reduced side to side scrolling, tooltips, and more detailed warnings for unusual inputs. Another planned enhancement could be a visual breakdown of costs using graphs or charts.These additions will make Grant Guide even more accessible and user-friendly while retaining the accuracy for real-world grants.

In addition to these improvements, we also want to refine what we already have. We understand that even though the logic is a lot of what this class goes over, the interface is really what engages people. Without an easy to follow multi-step wizard it can be easy for a user to just leave the website without even giving it a try. Feedback will be a big part of finishing this project on a high-note since it can be easy to get stuck in your own work. Grabbing different views can open us up to change that we never could have seen necessary.

9 Performance and Optimization Strategy

As Grant Guide evolved passed the basic interface we started with, performance becomes more of a concern especially if we want to keep on working on it in the future. Although the application is not computationally intensive, it deals with frequent database queries, dynamic calculations, live updates to the interface, and client-server connections that must retain responsive for users to get the most out of the multi-step budgeting wizard. Our performance strategy focuses on three areas: efficient data retrieval, unnecessary server work, and ensuring front-end calculations remain fast as the number of entries can grow very large.

The first major performance decision was to get rid of as much time as possible from the front-end computation. Since JavaScript can handle arithmetic operations quickly it made sense for us to not have to send every update back to the server. For example, when the user changes the effort percentage in our second step of the interface or adds a travel entry, the entire summary of costs is recalculated on the client side without performing any additional queries. This prevents the server from being way to overloaded with simple arithmetic requests, and ensures that the update still appear instantly on the screen. This responsiveness is very important for ensuring that the users can see the real time feedback as soon as changes are made.

To further optimize the data flow in the project, we designed the backend API to retrieve reference tables only once per page load. Tables like salaries, travel-profiles, and tuition schedules rarely change during the budgeting process, meaning that fetching the tables over and over again would just add server overhead for no reason. Instead, the interface can now load these tables at startup and stores them in memory through a state object. From this point all calculations can reuse this data, which reduces the number of requests made to the server.

All in all, the performance and optimization strategy for Grant Guide provide a responsive interface for users to use. And because of our strategy it also won't be hard to add more complex features to what we already have, and it will always be capable of handling increasingly more complex budget structures. By carefully balancing server-side efficiency with real time front-end calculations and smart data management, we have created an application that feels lightweight and not overwhelming to the user while also maintaining accurate, multi-year financial budgeting plans.

10 Reflection/Lessons Learned

This project has given us insight on what it's like to work in the field of databases and full stack projects. We quickly learned that to be successful in a project of this scale, it's essential to stay organized, structured, and consistent with the work schedule planned in advance. Early on, we discovered that progress on one area such as the front end was often dependent on the completion of another. Because of that, there were times when we worked intensely and made a lot of progress, and other times when we had to slow down and wait for other parts of the project to be completed. These experiences taught us the importance of time management, communication with team members, and pacing our work to keep all parts of the project aligned.

One of the biggest areas of growth for us came from learning how to collaborate and have Version Control through Github. At first, we were definitely hesitant to use it since neither of us had much experience with it. This made us altogether avoid using Github at first, but we quickly learned that this was unrealistic(especially after our lab project). Once we finally took it into account that

we needed to learn Github we were able to work simultaneously on different sections of the project without constantly waiting for each other to finish. However, this also added new challenges such as merging code that broke our working features. Through these new challenges we learned the importance to either keep separate branches of code or to have backups of our past working code.

In the end of Phase II, the project is coming along at a steady rate and we are fully confident that we will finish the project by the time it is due. Since, we are confident in these regards we hope that Budget Builder won't just meet the requirements but will be polished enough that the others could use it as a practical tool or as a model for future students that come through this class.

11 Plan and Timeline

This project was started in the fall semester of 2025 at the University of Idaho. The development is divided into multiple phases, with this report being the end of phase three. Phase three required a prototype of the application with most features working correctly and an user interface built to go with it. The project will continue until the end of the semester when Grant Guide will be complete.

October 2nd: On this date we gave our overview of our presentation in class using Google Slides. We decided to use HTML/CSS and Javascript for the front end, node.js for the back end and MySql for the database.

October 30th: We showcased the working prototype of our application in class. Currently, about half or so of Grant Guide is working and fully functioning while the other half has front end elements implemented but no functionality.

November 21: By this time, our system was significantly improved with complete functionality. The application will be very easy for users to navigate through completing an acceptable budget for federal and state grants. These features are present in the document and were presented to the class.

December 9th: This will be the final part of our process. At this stage of development Grant Guide was 100 percent done. There was no errors at this time and we were ready to present.

12 Conclusion

To conclude, Grant Guide will be a system designed to help researchers build year long budgets for grants. Then, the front end part of this full stack project will walk users step by step through the process so it is not complicated at all.

Researchers can log into our system and create or manage projects, track the budget details, and export budget reports as an Excel spreadsheet.

Overall, Grant Guide will make the process as efficient and reliable as possible to save time, and improve compliance with funding institutional requirements.

13 Installation and Setup Instructions

All source code and data for this project are available in a public Github repository, linked at the bottom of this section. To run the system locally, users must have Node.js installed on their machine. After cloning the repository, dependencies can be installed using npm install. The backend server must then be started by running node server.js, which launches the API required for authentication, data retrieval, and budget calculations. Lastly, upload the schema to mySQL workbench for the database to be loaded in. Once the server is running, the web interface can be accessed through your localhost.

Link: `https://github.com/danny-g04/grant_guide`.